How Small Can VLA-JEPA Get? Post-Training Compression of a Robot Policy on LIBERO

Noah Sprenger Fernando Nogueira
CSI 5341, University of Ottawa Code: https://github.com/seofernando25/csi5341-vla-jepa

***Abstract*—VLA-JEPA couples a 2B-parameter vision–language model with a flow-matching action head and reaches 95% success on LIBERO-Spatial, with a measured peak of 5.6 GB allocated GPU memory at batch size one. We test whether post-training compression keeps that success on this suite. Quantizing only the language-decoder linear layers to NF4 cuts peak memory by 36.9% with a paired success change of +0.8 points (95% CI −0.6 to +2.2, 500 trials). LLM.int8 cuts 23.3% and misses our 25% target, which equals the ideal INT8 saving for the decoder weights. A sweep of 201 compression recipes finds that the language decoder is the precision bottleneck, while vision and action weights survive 2-bit quantization. Two joint recipes keep 96.8–98.0% success at 1.38–1.51 GB (73–75% less memory) and use no calibration data. Under the stock execution path every compressed policy is 1.2–2.0× slower, because that path is launch-bound. A CUDA-graph engine speeds up B16 itself from 77.5 to 28.5 ms; the NF4 policy C2 then runs at 31.6 ms in 1.51 GB with 98.0% success. Offline action divergence on fixed frames predicts gross failure ($\rho = -0.67$, 163 recipes) but cannot rank near-lossless ones. Replacing the backbone with SmolVLM2-500M failed (2.2%).**

I. Introduction

Vision–language–action (VLA) policies map camera images and an instruction to robot actions [1], [2]. VLA-JEPA [3] adds a V-JEPA 2 latent world model [4] during training to a Qwen3-VL-2B backbone [5] and a flow-matching action head. Its weights alone occupy 5.5 GB in BF16, which excludes many embedded GPUs. We ask how far the *trained* policy can be compressed without retraining, and what breaks first.

Our proposal stated one hypothesis. The feedback asked for justified thresholds and more depth, so we test four:

- **H1** Quantizing the language-decoder linear layers reduces peak inference memory by ≥25% and lowers LIBERO-Spatial success by ≤5 points.
- **H2** Tolerance to compression differs by module: the language decoder is more sensitive than the vision encoder or action head.
- **H3** At batch size one, lower memory does not imply lower latency.
- **H4** The change in predicted actions on fixed observations predicts closed-loop failure, so it can screen recipes without rollouts.

Thresholds. The 25% memory target is the ideal saving of 8-bit decoder weights: 1.41B linear parameters lose one byte each, 1.41 GB of a 5.61 GB peak, or 25.1% (Fig. 2). **H1** therefore asks whether runtime overhead consumes the theoretical saving. The 5-point bound allows the 5% baseline failure rate to at most double. With 500 paired trials, the 95% interval on a paired difference spans 2–4 points, narrow enough to separate a 5-point drop from no change.

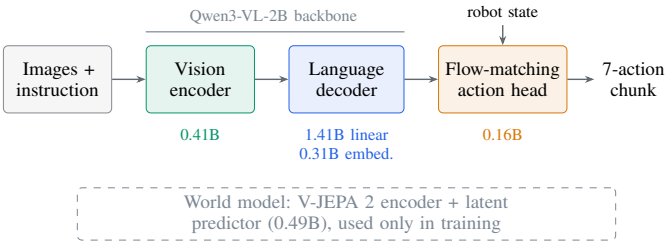


Fig. 1. VLA-JEPA inference path with parameter counts. Colored modules are the compression targets; mergers connect vision features to the decoder. The world model is loaded by default but unused at inference.

Contributions. (i) A matched 500-trial evaluation of INT8 and NF4 decoder quantization with paired statistics. (ii) A module-level sensitivity map over 201 recipes and 21 component domains. (iii) Two deployable recipes below 1.6 GB with reserved-trial confirmation. (iv) A failure analysis linking offline action change to task success.

II. Related Work

VLA policies. OpenVLA [1] discretizes actions from a 7B language model; π_0 [2] and VLA-JEPA use flow matching. SmolVLA [6] trains a 0.45B policy from scratch on a SmolVLM backbone [7]. We compress an existing policy instead of training a small one.

Quantization. LLM.int8 [8] keeps outlier features in FP16; NF4 [9] stores 4-bit normal-float weights with double-quantized scales. GPTQ [10] and AWQ [11] calibrate weights on sample inputs; HQQ [12] quantizes without calibration down to 2 bits. QuantVLA [13] adapts post-training quantization to VLA action experts.

Pruning. Wanda [14] and SparseGPT [15] prune individual weights; structured pruning removes channels, heads or layers, and EcoVLA [16] applies it to VLAs. We rank layers by linear CKA [17] between block input and output. Unstructured masks save no memory without sparse kernels, so we report memory only for structured and packed formats.

III. Method

Policy. We use the released LIBERO checkpoint of the LeRobot port [18] (`VLA-JEPA-LIBERO`), pinned by revision (Fig. 1). The policy predicts seven actions with four Euler flow steps and executes all seven before replanning. Nothing in this report retrains Qwen.

TABLE I

EXPERIMENTAL DESIGN. ALL ROWS SHARE THE CHECKPOINT, TASK SUITE, STATES, SEEDS, ACTION HORIZON, GPU AND SOFTWARE VERSIONS; E7 ALSO CHANGES THE FLOW STEPS.

ID	Question	Varied	Sample	Analysis and decision rule
E1 (**H1**)	Does decoder quantization keep success?	Decoder linear precision: BF16, INT8, NF4	500 reserved rollouts per arm	Paired bootstrap; pass if memory reduction ≥25% and $\Delta \geq -5$ pp
E2 (**H2**)	Which modules tolerate compression?	Module × method × strength; 201 recipes	80 fixed frames; 10-trial smoke (163); 100-trial development (24)	Development gate ≥77/100 (B16 82 −5); selection frozen before E3
E3	Do joint recipes hold on reserved trials?	Two frozen joint recipes	500 reserved rollouts each	Wilson and paired intervals; report regardless of outcome
E4 (**H3**)	Does lower memory mean faster?	All E1/E3 arms	3×500 timed predictions	Median and p95 latency vs. peak memory
E5 (**H4**)	Where and why do arms fail?	—	Per-episode outcomes; 80-frame action change	Paired transitions per task; Spearman ρ
E6	Can a smaller backbone replace Qwen?	SmolVLM2-500M + adapter, 10k fine-tuning steps	500 reserved rollouts	Same as E1
E7 (**H3**)	Is the slowdown intrinsic to quantization?	Execution engine (stock or CUDA graph), flow steps (4 or 1), dense or packed NF4 weights	3×500 timed predictions per variant; 500 reserved rollouts for C2	Same as E1; the one-step setting comes from the E2 smoke screen

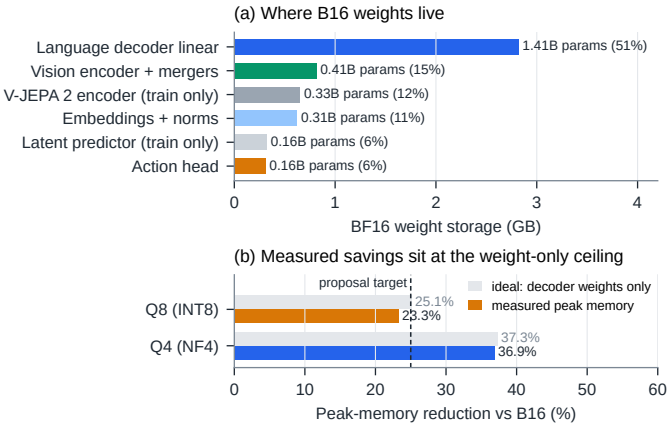


Fig. 2. (a) B16 parameters by module. (b) Weight-only savings ceiling for quantizing the decoder linear layers versus the measured peak-memory reduction.

Compression arms. *Study A* quantizes every `nn.Linear` in the 28-layer Qwen decoder after the checkpoint loads: Q8 with LLM.int8 (outlier threshold 6) and Q4 with NF4 (double quantization, BF16 compute). Every other module stays BF16. *Study B* applies 201 recipes to individual modules and joint deployments: bitsandbytes INT8/NF4, HQQ 2–4 bit, GPTQ, AWQ-style scaling, TorchAO W8A8, magnitude/Wanda/SparseGPT sparsity, and structured removal of MLP channels, attention heads, layers and tokens. Calibrated methods use 20 development frames; no recipe sees reserved trials. Joint deployments also drop the unused world model. Some sweep recipes restrict the vocabulary to the 53 token IDs of the LIBERO instructions; we report none of them as a result.

Protocol. Table I lists the experiments. All runs use one RTX 3090, PyTorch 2.11, Transformers 5.5.4 and bitsandbytes 0.50.2. Rollouts use the ten LIBERO-Spatial tasks [19] and the 50 official initial states per task, with fixed per-episode seeds and a 280-step limit. Development trials use 10 different states per task, disjoint by hash. Success intervals are Wilson intervals [20]. Differences from B16 use a task-stratified paired bootstrap (10,000 resamples) over identical states and seeds. Latency is the synchronized observation-to-action time over 3×500 predictions after 20 warm-ups and excludes simulation. Memory is the peak allocated by PyTorch's CUDA allocator, in GB (10^9 bytes).

IV. RESULTS

A. Decoder quantization (*H1*)

Both quantized decoders keep success: neither paired interval excludes zero, and both exclude the 5-point bound (Table II). Q4 passes **H1** with 36.9% less memory. Q8 fails on memory: 23.3% against a 25.1% ceiling. The measured savings sit 1.8 and 0.4 points below the weight-only ceilings for INT8 and NF4 (Fig. 2b). Activations, quantization scales and LLM.int8's FP16 outlier path are the likely sources of the gap; we did not profile them separately. The decoder holds only 51% of the weights, so decoder-only quantization cannot save more than about 37%. Larger savings require compressing the other modules.

Our B16 reaches 95.0%, against 96.2% reported for LIBERO-Spatial [3]. We use the LeRobot port and our own evaluator, not the authors' evaluator.

B. Where compression fails (*H2*)

Fig. 4 shows 10-trial smoke results; B16 scores 8/10 on the same states. The language decoder is the precision bottleneck.

TABLE II

RESERVED-TRIAL RESULTS (500 ROLLOUTS EACH, RTX 3090). Δ: PAIRED SUCCESS CHANGE FROM B16 WITH 95% INTERVAL. GAIN/LOSS: STATES WHERE ONLY THE VARIANT OR ONLY B16 SUCCEEDS. STUDY B HAS ITS OWN B16 CONTROL (475/500, IDENTICAL PER-EPISODE OUTCOMES; MEDIAN 77.5 MS).

	Configuration	Success % [CI]	Δ pp [CI]	Gain/loss	Peak GB (−%)	Weights GB	Median (p95) ms	**H1**
A	B16: BF16	95.0 [92.7, 96.6]	—	—	5.61	5.53	79.1 (84.3)	—
A	Q8: LLM.int8 decoder	94.8 [92.5, 96.4]	−0.2 [−1.2, +0.8]	3/4	4.30 (23.3)	4.13	157.4 (161.4)	**fail**
A	Q4: NF4 decoder	95.8 [93.7, 97.2]	+0.8 [−0.6, +2.2]	8/4	3.54 (36.9)	3.44	93.6 (97.9)	**pass**
B	C1: NF4 lang., HQQ2 vision/action	96.8 [94.9, 98.0]	+1.8 [−0.2, +3.8]	19/10	1.38 (75.4)	1.29	142.9 (145.5)	**pass**
B	C2: NF4 all, 1 step, graph†	98.0 [96.4, 98.9]	+3.0 [+1.4, +4.8]	18/3	1.51 (73.1)	1.34	31.6 (32.0)	**pass**
	S500: SmolVLM2-500M	2.2 [1.2, 3.9]	−92.8 [−94.8, −90.6]	—	3.30 (41.1)	2.30	1269.7 (1277.8)	**fail**

†No pruning and no calibration data; INT8 embedding table (full vocabulary); one flow step instead of four; CUDA-graph engine with fused kernels and packed weights. Peak is allocated memory; the allocator reserved 2.71 GB. C1–C2 also remove the unused world model.

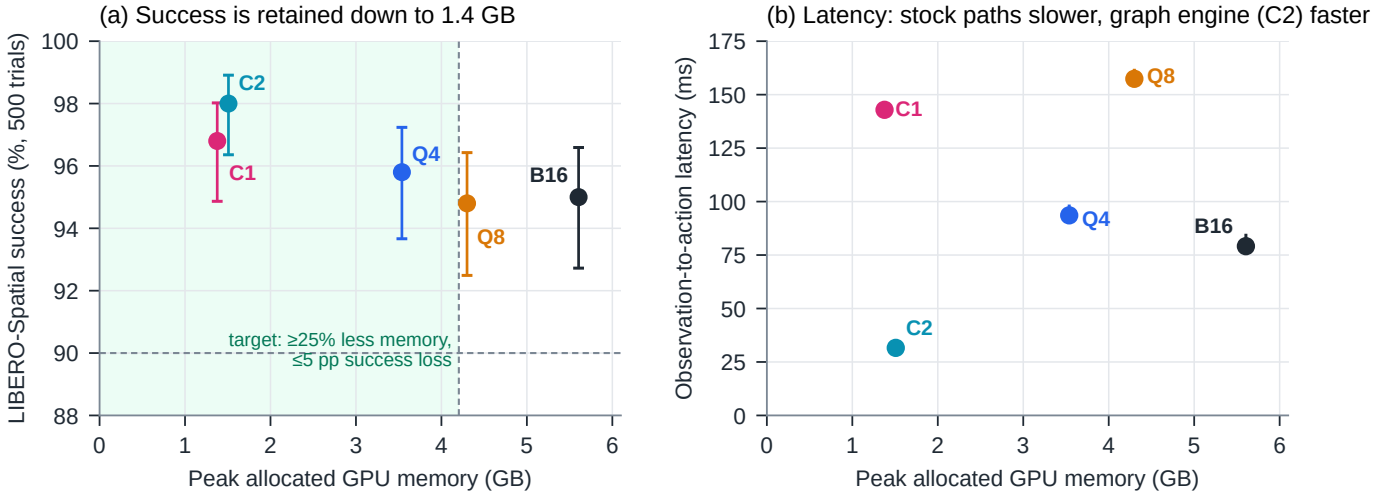


Fig. 3. Reserved-trial trade-offs. (a) Success with 95% Wilson intervals; the green region meets **H1**'s memory and success targets. (b) Median latency with p95 tick. Q8/Q4 come from Study A, C1–C2 from Study B (C2 with the graph engine); both B16 controls produce identical outcomes.

Its best 3-bit and 2-bit variants score 7/10 and 1/10, and 3-bit results vary with the quantization axis (7/10 versus 0/10). Vision blocks and action MLPs keep 8/10 at 2 bits under every tested variant. Structure behaves the same way. Removing 8 of 28 language layers or 50% of its key–value heads scores 0/10 and 2/10, respectively. Vision keeps 8/10 with 50% of channels or heads removed and with 8 of 24 layers removed. Action modules degrade gradually. These smoke screens are coarse. At 100 development trials, removing 75% of action attention heads scores 69/100 and fails the 77/100 gate, although its smoke score was 7/10. Combining individually safe cuts also fails: joint channel and depth reductions score 70/100 and 65/100.

These results guided the joint recipes. C1 keeps NF4 in the language decoder and uses 2-bit HQQ in the vision and action modules, with an INT8 embedding table: 1.29 GB of weights and 1.38 GB peak memory. C2 uses NF4 everywhere with an INT8 embedding table. Both pass **H1** on reserved trials (Fig. 3a).

C. *Memory is not speed (**H3**)*

Under the stock execution path every compressed policy is slower than B16 (Fig. 3b). Q8 doubles latency (1.99×), Q4 costs 1.18×, and C1 costs 1.84×. The cause is launch overhead, not arithmetic. A profile of one stock prediction (`launch_profile.json`) shows B16 launching 4,989 kernels with the GPU busy 36 ms of 79 ms. Q4 adds a dequantization kernel per linear layer (5,576 launches). LLM.int8 launches 9,302 kernels and synchronizes with the host 691 times (B16: 118): its 192 `nonzero` calls (B16: 11) force the host to wait for the GPU.

We therefore wrote a CUDA-graph engine (`evaluation/fastpath.py`). It precomputes everything that depends only on the instruction, captures one graph per prompt, fuses elementwise kernels with `torch.compile`, and dequantizes NF4 with one Triton kernel that matches bitsandbytes bit for bit. Table III separates its effect from the compression. The engine alone takes B16 from 77.5 to 28.5 ms. On that fair baseline NF4 weights cost latency: 36.8 ms with four flow steps against 28.5 ms, in exchange for 3.8× less peak memory (1.51 versus 5.70 GB). Dense bf16 copies of the NF4-rounded weights remove the dequantization (24.7 ms) at 5.45 GB. C2 combines NF4 on all modules, an INT8 embedding table, no pruning, no calibration data and one flow step; we chose these to remove task-specific components, not by tuning on outcomes. It runs at 31.6 ms in 1.51 GB. The engine captures one graph per instruction; priming ten

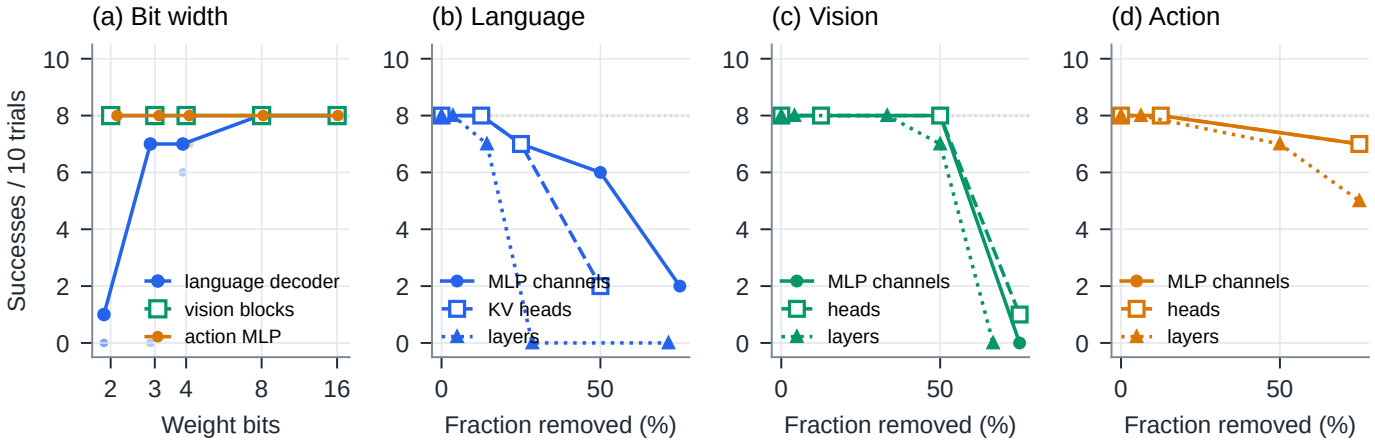


Fig. 4. Module sensitivity (10 trials per point; dotted line: B16 at 8/10). (a) Best tested variant per bit width; faint points show other variants. (b–d) Structured removal by module. Missing points were not tested because adjacent strengths already failed.

TABLE III
LATENCY ATTRIBUTION (PIPELINE MEDIAN, P95 IN PARENTHESES; 3 × 500 PREDICTIONS). PEAK IS ALLOCATED GPU MEMORY. SUCCESS: OF 500 RESERVED TRIALS; DASHES WERE NOT RUN.

Variant	Steps	Median (p95) ms	Peak GB	Success
B16, stock path	4	77.5 (80.5)	5.61	475
B16, graph engine	4	28.5 (28.9)	5.70	474
B16, graph engine	1	24.8 (25.1)	5.70	—
NF4 all, graph, packed	4	36.8 (37.7)	1.51	486
NF4 all, graph, dense	1	24.7 (24.9)	5.45	—
NF4 all, graph, packed (C2)	1	31.6 (32.0)	1.51	490

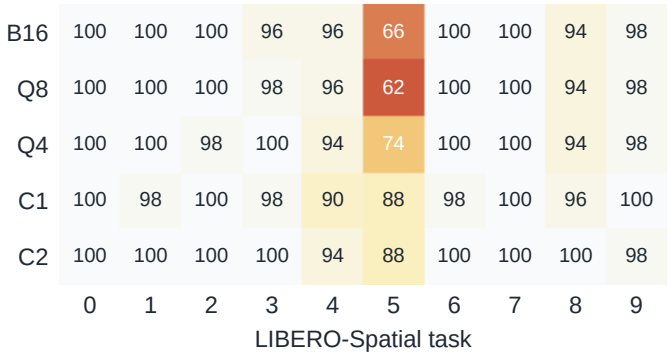


Fig. 5. Per-task success (%, 50 reserved trials per cell). Task 5 accounts for most failures and most differences between arms.

prompts took 109 s, which the latencies exclude. **H3** holds for the stock path: memory and latency are independent, and execution sets latency.

*D. Failure analysis (**H4**)*

Failures are timeouts in one task. Rollouts are deterministic given state and seed: the two B16 runs agree on all 500 episodes. All 25 B16 failures hit the 280-step limit; none crashed. Seventeen occur in task 5, "pick up the black bowl on the ramekin" (Fig. 5). Quantization mostly inherits these failures: 22 of Q8's 26 and 17 of Q4's 21 failed states also fail under B16.

Task 5 decides the aggregate. C1's net gain of 9 successes is +11 in task 5 (33→44/50) and −2 across the other nine tasks. C2's net gain of 15 is +11 in task 5 and +4 elsewhere. A task where small action perturbations flip outcomes can create apparent "improvements" from compression, so neither gain is evidence that quantization improves the policy. Two controls locate the effect (Table III). B16 through the graph engine scores 474/500 (−0.2 points, 95% CI −1.2 to +0.8), so the engine does not change success. The same NF4 weights with four flow steps score 486/500 (+2.2, +0.8 to +3.8), and one step adds four successes, within noise. The gain therefore appears with quantized weights, not with the engine or the solver. We report it as unexplained.

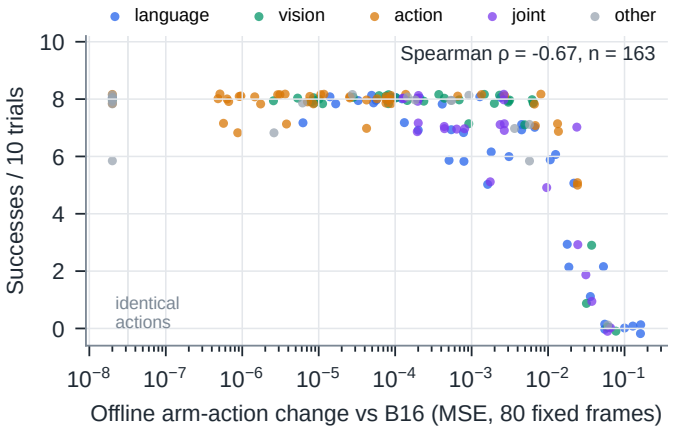


Fig. 6. Offline action change on 80 fixed development frames versus 10-trial success, for 163 recipes (jittered vertically). Recipes with identical actions sit at the left edge.

Offline divergence screens, but does not rank. For each recipe we measured the mean squared change in physical arm commands against B16 on 80 fixed frames, which needs no simulator. Across 163 recipes it correlates with smoke

success ($\rho = -0.67$, $p < 10^{-21}$; Fig. 6). Below 10^{-4}, 64 of 65 recipes score ≥7/10; above 10^{-3}, only 30 of 62 do. Among the 24 recipes that reached development trials, the correlation is weak and not significant ($\rho = -0.27$, $p = 0.21$). **H4** holds for screening but not for ranking: closed-loop trials remain necessary among near-lossless recipes.

E. Negative result: smaller backbone

We replaced Qwen with SmolVLM2-500M [7] plus a trained adapter and fine-tuned the adapter, action head and predictor for 10,000 steps. It reached 2.2% on reserved trials and was 16× slower than B16, because SmolVLM tiles each image into 17 crops of 512 pixels. A later study fixed an image-scaling bug and adapted all decoder layers. It reached 31/100 development successes against 82/100 for B16. Within our budget, compressing the trained policy beats replacing its backbone.

V. Recommendations

1) **For a single-GPU deployment with a memory limit, start with Q4.** It uses a standard bitsandbytes configuration, saves 37% of peak memory, shows no detectable success loss, and costs 18% latency on the stock path.
2) **For latency, run a CUDA-graph engine.** It makes B16 itself 2.7× faster (28.5 ms). C2 runs at 31.6 ms in 1.51 GB; B16 with one flow step runs at 24.8 ms in 5.7 GB.
3) **Below 1.6 GB, use C1 or C2.** Both keep the full vocabulary and use no calibration data.
4) **Avoid LLM.int8 at batch size one.** It is slower than NF4 and saves less memory.
5) **Spend precision on the language decoder**, and compress vision and action modules first.
6) **Screen with offline action change, confirm with ≥500 paired trials.** Ten or even 100 trials cannot rank near-lossless recipes, and per-task paired transitions expose single-task effects such as task 5.

VI. Limitations and Conclusion

We evaluated one suite (LIBERO-Spatial), one checkpoint, one GPU and one seed per state. Success on Object, Goal or Long suites, on other hardware, and on a real robot is untested. Development and smoke selection used 10–100 trials, so sensitivity curves are coarse. Neither joint recipe uses pruning, calibration data or a task-specific vocabulary. We chose every arm on smoke and development states, never on reserved trials, but the same 500 reserved states score all arms with no multiple-comparison correction. The graph engine assumes one observation, two equal-size images and unpadded prompts per call, and its fused kernels change bf16 rounding, so we measured success through the engine. Compressed archives are rebuilt from the original checkpoint plus a recipe and are not standalone Hugging Face checkpoints.

Post-training compression is effective for VLA-JEPA. NF4 decoder quantization meets our memory and success targets. Joint recipes keep ≥96% success in under 1.6 GB, a 3.7–4.1× reduction. The language decoder limits further compression, and a single hard task drives most differences between arms. Latency rises under the stock path; a graph engine removes the overhead for every variant, and C2 runs at 31.6 ms in 1.51 GB. Fused 4-bit GEMM kernels and evaluation on longer-horizon suites are the next steps.

References

[1] M. J. Kim *et al.*, "OpenVLA: An open-source vision-language-action model," in *CoRL*, 2024.
[2] K. Black *et al.*, "π_0: A vision-language-action flow model for general robot control," *arXiv:2410.24164*, 2024.
[3] J. Sun, W. Zhang, Z. Qi, S. Ren, Z. Liu, H. Zhu, G. Sun, X. Jin, and Z. Chen, "VLA-JEPA: Enhancing vision-language-action model with latent world model," *arXiv:2602.10098*, 2026.
[4] M. Assran *et al.*, "V-JEPA 2: Self-supervised video models enable understanding, prediction and planning," *arXiv:2506.09985*, 2025.
[5] Qwen Team, "Qwen3-VL-2B-Instruct," Hugging Face model card, 2025.
[6] M. Shukor *et al.*, "SmolVLA: A vision-language-action model for affordable and efficient robotics," *arXiv:2506.01844*, 2025.
[7] A. Marafioti *et al.*, "SmolVLM: Redefining small and efficient multimodal models," *arXiv:2504.05299*, 2025.
[8] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *NeurIPS*, 2022.
[9] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *NeurIPS*, 2023.
[10] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in *ICLR*, 2023.
[11] J. Lin *et al.*, "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *MLSys*, 2024.
[12] H. Badri and A. Shaji, "Half-quadratic quantization of large machine learning models," https://github.com/dropbox/hqq, 2023.
[13] "QuantVLA," arXiv:2602.20309, 2026.
[14] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter, "A simple and effective pruning approach for large language models," in *ICLR*, 2024.
[15] E. Frantar and D. Alistarh, "SparseGPT: Massive language models can be accurately pruned in one-shot," in *ICML*, 2023.
[16] "EcoVLA," arXiv:2602.00780, 2026.
[17] S. Kornblith, M. Norouzi, H. Lee, and G. Hinton, "Similarity of neural network representations revisited," in *ICML*, 2019.
[18] R. Cadene *et al.*, "LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch," https://github.com/huggingface/lerobot, 2024.
[19] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," in *NeurIPS Datasets and Benchmarks*, 2023.
[20] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, pp. 209–212, 1927.